Assignment 3 Report

Pipelined RV32I Core and LLM Verification Agent

Student	Dhanush Adithya Lakkapuram Ragupathy
UIN	837004069

Summary

The completed processor passed all 13 supplied correctness tests. The verification agent produced a 22-test mutation-sensitive suite, raised coverage from 20 of 21 to 21 of 21 bins with two targeted AUIPC tests, and generated eight assertions that remained silent on all 24 final tests.

Task 1 Pipelined core completion

The starting datapath already contained the five pipeline stages and registers but left three control regions incomplete. Six supporting modules were reused from Assignment 2 without changing their file or module names: alu, branch_unit, controller, extend, memory_access, and regfile.

Forwarding unit

For each Execute-stage source, the forwarding logic compares Rs1E or Rs2E against valid destination registers in Memory and Writeback. A matching Memory producer selects 2'b10. Otherwise, a matching Writeback producer selects 2'b01. If neither matches, the register-file value is used. Memory has priority because it contains the newer value when both stages target the same register. The Rd != 0 guard prevents an internal result nominally addressed to x0 from being forwarded.

Load use hazard detection

A load in Execute cannot supply its memory result to the immediately following Decode instruction in time. LoadUseHazard is asserted when ResultSrcE marks a load, RdE is nonzero, and RdE matches either Rs1D or Rs2D. Fetch and Decode hold for one cycle while Execute receives a bubble. Checking both source fields covers arithmetic, stores, and comparisons that consume the loaded value in either operand.

Redirect and flush control

A taken branch, JAL, or JALR resolves in Execute through PCSrcE. RedirectE selects RedirectTargetE as the next fetch address, flushes Decode, and flushes the younger instruction entering Execute. A load-use event stalls Fetch and Decode and also flushes Execute. JALR targets clear bit zero in the datapath as required by RV32I.

Correctness evidence

Run	Passing	Coverage	Uncovered
Supplied Task 1 suite	13 of 13	20 of 21 (95 percent)	OP_AUIPC

The supplied tests exercised EX-to-EX and MEM-to-EX dependencies, dependency chains, load-use stalls through both sources, PC holding, taken and untaken branches, JAL and JALR links and redirects, load-byte sign extension,

shifts and logic, and x0. Every program reached its expected signature. The remaining AUIPC bin reflected test content rather than a failing instruction result.

Task 2 LLM verification agent

Provider integration and output contract

The reported runs held the model and sampling settings fixed at protected.Claude-Opus-5, temperature 0.0, and max_tokens 16,384 through TAMU_AI_CHAT.

Generated test format

The model returned structured JSON descriptions of RV32I programs. These are neither SystemVerilog testbenches nor assertions. Each instruction object is syntax-checked and encoded into a 32-bit word. The agent appends a signature store to address 0x100, runs the reference instruction-set simulator to compute the expected signature, and writes the words as a .hex program. Invalid operations, registers, immediates, control offsets, duplicate names, or programs that never reach the signature store are rejected.

Oracle comparison and mutation analysis

The selected RTL is compiled with Icarus Verilog. For each program, the testbench runs the core and compares its signature against the reference result. The suite is also run on five deliberately corrupted cores. A mutation is detected when at least one test fails on that mutant while the suite passes on the selected core. This turns broad prompting into measured evidence for specific bug families.

Mutation family	Representative injected defect	Result
Forwarding	Writeback incorrectly beats the newer Memory producer	Detected
x0 integrity	Memory forwarding is allowed for destination x0	Detected
Load use	Load-use stall is disabled	Detected
Flush and redirect	Execute flush after redirect is disabled	Detected
Branch and jump target	JALR target bit zero is not cleared	Detected

Task 2 result

Eight deterministic seed programs supplied a safety net. One LLM call produced fourteen valid directed programs, giving 22 total. All 22 passed the correct core and the suite detected all five mutation families. No repair call was needed. Coverage remained 20 of 21 because neither the seeds nor the first generated batch executed AUIPC.

Task 3 Coverage guided generation

Coverage measurement

Coverage contains 21 fixed bins: nine instruction groups, seven hazard scenarios, and five control paths. The instrumented testbench records Execute-stage opcodes, forwarding selections, stalls, flushes, and control outcomes. Only bins from passing tests count. Cumulative coverage is the union of every bin hit at least once across the suite. Per-test JSON entries show which programs contributed each bin.

Dimension	Bins
Instruction groups (9)	OP_REG, OP_IMM, OP_LOAD, OP_STORE, OP_BRANCH, OP_JAL, OP_JALR, OP_LUI, OP_AUIPC
Hazards (7)	fwdA_mem, fwdA_wb, fwdB_mem, fwdB_wb, no_forward, load_use_stall, flush
Control paths (5)	branch_taken, branch_not_taken, jal, jalr, no_control

Feedback loop

The loop first runs the existing 22-test suite with coverage instrumentation. If bins are missing, the user prompt includes every existing passing test and its observed bins, the cumulative hit list, the missing list, earlier attempt outcomes, the fixed bin definitions, and the JSON instruction schema. The model is asked for only one to three minimal additions. Candidates are parsed, checked by the reference simulator, run on the RTL, and retained only when passing tests add coverage. The loop remeasures and stops at full coverage or when the shared call limit is exhausted.

Measurement	Passing tests	Bins	Percent	Missing
Before feedback	22	20 of 21	95.24	OP_AUIPC
After feedback	24	21 of 21	100.00	None

OP_AUIPC was the only difficult bin because the functional and hazard tests had no reason to execute a PC-relative upper-immediate operation. One coverage-feedback call generated auipc_pc_relative_signature and auipc_delta_with_jal_link. Both passed, propagated AUIPC-derived values to their signatures, and raised coverage to 21 of 21. The stop reason was coverage_complete.

Task 4 Assertion generation

The assertion prompt supplied Claude with the exact module template, invariant specification, and an Icarus-compatible coding contract. It required immediate assertions in clocked blocks, reset guards, the unchanged module interface, and eight stable failure names. One model call produced the accepted results/assertions.sv.

Invariant group	Checks
x0 read value	Rs1D == 0 implies rd1D == 0, and Rs2D == 0 implies rd2D == 0
No forwarding into x0	An x0 Execute source implies its forwarding select is 00
Memory producer exists	Each 10 select requires RegWriteM, matching RdM, and RdM != 0
Writeback producer exists	Each 01 select requires RegWriteW, matching RdW, and RdW != 0

Validation strategy

- Static validation rejects a changed interface, unsupported property syntax, missing reset guards, or missing and duplicate stable assertion names.
- A synthetic activation probe deliberately violates each invariant and confirms that every required name can fire. All eight were observed.
- The assertion-enabled correct-core suite checks signatures and false positives. Compilation passed, all 24 tests passed, and no ASSERT_FAIL line appeared.

The activation probe matters because an assertion can compile and remain silent if its condition is ineffective. The correct-core run provides the complementary check: useful assertions must fire under injected violations and remain silent during valid behavior.

Task 5 Experiment accounting and reproducibility

Task and purpose	Calls	Prompt Input tokens	Completion Tokens	Total
Task 2 directed tests	1	1,213	9,021	10,234
Task 3 coverage feedback	1	4,798	915	5,713
Task 4 assertions	1	2,077	1,606	3,683
Total	3	8,088	11,542	19,630

The maximum was 20 LLM calls across the cumulative workflow. Each request increments the counter before network access, so failed requests also consume budget. Task 2 used one call, Task 3 began with that saved count and used one more, and Task 4 began at two and used one. The final experiment used 3 of 20 calls. Transcripts record provider, requested and returned model, prompts, content, settings, timing, status, finish reason, and token usage.

Analysis, limitations, and improvements

Difficult behaviors

Forwarding priority is subtle because both Memory and Writeback can match the same source in one cycle, and choosing the older value can still pass simpler dependency tests. Load use differs from ordinary RAW hazards because forwarding alone cannot make the data available soon enough. Redirects require two coordinated actions: select the target and kill both younger instructions. JALR adds the easily missed bit-zero rule. The directed suite makes each detail affect a signature rather than merely executing the relevant instruction.

What was caught and what remains

The suite caught every injected representative defect, so there were no missed mutation families in the reported experiment. This does not prove the absence of all RTL bugs. Five source-level mutants sample the required families but do not exhaust wrong comparators, malformed immediates, reset behavior, memory alignment, simultaneous stall and redirect corner cases, or arbitrary multi-cycle interactions. Similarly, 21 of 21 coverage means every defined bin occurred at least once. It is not exhaustive state or cross coverage.

Effect of the model and provider

Claude Opus 5 produced fourteen valid directed tests in the first Task 2 response, two useful AUIPC tests from one targeted coverage response, and a valid eight-assertion module from one assertion response. Temperature 0.0 and strict JSON and SystemVerilog contracts reduced parsing ambiguity. The TAMU gateway also exposed token usage and model identifiers, making the run auditable. The agent did not trust model output directly: encoders, the reference simulator, RTL simulation, mutation results, coverage gain, structural checks, and activation probes decided what was accepted.

Improvements with more time

- Add cross coverage for opcode-by-hazard and redirect-by-forwarding combinations, plus repeated hits rather than single-bin presence.
- Expand mutation operators across both operands, branch conditions, immediate formation, pipeline-register enables, and simultaneous hazard priorities.
- Minimize accepted programs automatically while preserving their signature failure and coverage contribution.
- Add forwarding-priority assertions if more internal signals become available, and run assertions against targeted mutants as well as synthetic probes.
- Record tool versions and stable relative paths to improve portability between the local workspace and Olympus.

These improvements would strengthen the acceptance filters and coverage model while keeping the reported provider, model, temperature, and token setting unchanged.

Reproduction commands

```
conda env create -f environment.yml
conda activate ecen689-a3
cp env.example .env    # then set TAMUS_AI_CHAT_API_KEY
bash scripts/run_task1.sh
bash scripts/run_task2.sh
bash scripts/run_task3.sh
bash scripts/run_task4.sh
bash scripts/verify_task4.sh
```

The same model and settings should be retained when reproducing the reported experiment. A full rerun of Tasks 2 through 4 replaces agent-managed outputs, so the final report and submitted results should be checked together after the run.

Run Evidence map

Evidence	Location
Task 1 terminal record	run_artifacts/task1_terminal_output.txt
Task 2 test and mutation summary	results/run_records/run_summary.json and mutation_results.json
Coverage before and after	results/coverage.json and results/run_records/coverage_summary.json
Assertion generation and validation	results/assertions.sv and results/run_records/assertion_summary.json
Prompts responses and usage	results/run_records/calls, coverage_calls, and assertion_calls